oom-watch — Incident-Response Runbook

What to do when issues recur

OOM-Protect maintainers

2026-04-30

oom-watch — Incident-Response Runbook

This document is the **on-call playbook** for OOM-Protect. Every scenario it covers is one we have hit in production or during deployment. Each section follows the same shape so you can scan fast at 3 a.m.:

- **Symptom** — what you observe.
- **One-command diagnosis** — confirms the root cause.
- **Remediation** — exact recovery sequence.
- **Won't recur because** — the regression test or Challenge that locks the fix in.

Keep this document open while operating the daemon. The Constitution (`Constitution.md`) requires that every regression-class incident here also has a corresponding Challenge in `challenges/` — if you hit something not listed, file an issue and write the regression test before declaring it fixed.

Quick triage decision tree

```
Did the system OOM-cascade (user session died, processes destroyed)?
  └── §1  Production OOM cascade

Otherwise, is the oom-watch daemon healthy?
  ├── systemctl is-active oom-watch.service ⇒ failed
  │     ├── code=exited, status=2  → §2  Bad config
  │     ├── 'Start request repeated too quickly'  → §3  Restart throttle
  │     ├── code=exited, status=1, journal: 'atop binary not found'  → §4  atop missing
  │     └── 'atop sample failed: atop exit -1'  → §5  Sandbox blocking
  ├── systemctl is-active oom-watch.service ⇒ active(running) but no reports
  │     └── §6  Daemon running, host calm (expected) OR §7 thresholds wrong
  └── /var/log/oom-watch/reports/ filling disk
        └── §8  Report rotation

Did a deployment fail mid-step?
  ├── 'error obtaining VCS status'  → §9  Cross-UID build
  ├── 'Unknown key … in section [Service]'  → §10 systemd directive drift
  ├── '_comment' in error  → §11 Stale installed config
  └── any other failure  → §12 Use the diagnostic bundler
```

For everything not listed, run `sudo make oomwatch-diagnose` and read the resulting `/tmp/oom-watch-diagnose-<ts>.log`. The 16 sections cover ~95 % of failure modes.

---

§1 Production OOM cascade

Symptom. Your session evaporates: terminals close, browser windows vanish, IDE quits silently, you may be returned to the display manager / login screen. After re-login: `journalctl -b -1` shows the prior boot's `oom-watch` reports timestamps in the seconds *before* the cascade, plus `dmesg` shows the kernel OOM-killer's victim list.

One-command diagnosis.

```
ls -t /var/log/oom-watch/reports/*-critical.md 2>/dev/null | head -1 | xargs less
```

The most recent CRITICAL report — written **before** the cascade — contains the top processes, `/proc/meminfo`, PSI scores, swap state, user-slice cgroup memory, and a 200-line journal tail. That is the forensic ground truth.

Remediation.

1. Read the CRITICAL report. Identify the top-mem process: usually one runaway (browser tab leak, JVM heap overrun, build farm, model training, etc.).
2. Run `make verify` to confirm the umbrella (oomd + cgroup limits + sysctls from `oom-hardening.sh`) is still applied.
3. Run `make verify-stress` (or the lighter `make oommemhog-build && bash challenges/challenge-real-pressure.sh`) to confirm pressure detection still fires WARN.
4. If the runaway was a known wrappable workload, run it under `oom-runner.sh` going forward (e.g. `oom-runner --preset claude -- claude`). The per-leaf cgroup means the runaway dies in isolation.

Won't recur because.

- `oom-hardening.sh` puts a 56 G `MemoryMax` and 48 G `MemoryHigh` cap on `user-.slice` so the user-manager itself can no longer be the OOM victim.
- `systemd-oomd` (configured by the same script) kills leaf cgroups before the kernel OOM-killer is reached, with `SwapUsedLimit=80%` and `MemoryPressureLimit=50%`.
- `oom-watch` writes a forensic CRITICAL report when `MemAvailable` drops below 5 % or PSI memFull avg10 exceeds 30 %, **before** the kill happens, so the next cascade leaves real evidence even if it does evade the umbrella.

If the cascade still happens despite all three layers, the umbrella was misconfigured or unloaded — re-run `make install` to reapply `oom-hardening.sh`.

---

§2 Bad config (`code=exited, status=2`)

Symptom. `systemctl status oom-watch.service` shows `Active: failed (Result: exit-code)`, exit code 2, and the journal contains `config: parse /etc/oom-watch/config.json: …` followed by a specific error.

One-command diagnosis.

```
/usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run
```

Returns exit 0 with `config OK` if the file is valid; otherwise prints the exact validator error (unknown field, threshold ordering violation, interval out of range, etc.) and exits 2.

Remediation.

Two paths, in order of preference:

```
# A. Auto-remediate via the deployer (preserves your config in a backup):
sudo make oomwatch-deploy

# Step 2b will detect the broken config, copy it to
# /etc/oom-watch/config.json.broken.<timestamp>, install the shipped
# example, re-validate, and restart the unit. If you had custom
# thresholds, recover them from the backup file.

# B. Manual edit (when the validator's complaint is something you can
# fix in place — e.g. swapping warn and critical values):
sudo nano /etc/oom-watch/config.json
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run
sudo systemctl reset-failed oom-watch.service
sudo systemctl restart oom-watch.service
```

Won’t recur because.

- `challenges/challenge-config-validation.sh` asserts the **shipped** `oom-watch.example.json` itself passes `-dry-run`, so a future commit that re-introduces an unknown field or bad value cannot land without the test going red.
- `install-and-verify.sh` step 2b validates the **installed** config before asking systemd to start the unit — a bad edit produces the precise validator error, not a cryptic systemd exit code.

---

§3 Restart throttle (`Start request repeated too quickly`)

Symptom. `systemctl status oom-watch.service` shows `Failed with result 'exit-code'` and the journal contains `Start request repeated too quickly`. The unit refuses to start even after you fix the underlying cause.

One-command diagnosis.

```
journalctl -u oom-watch.service -n 5 --no-pager | grep -i 'restart counter'
```

A counter of ≥ 3 within the `StartLimitIntervalSec=60s` window means systemd has given up restarting the unit until you reset it.

Remediation.

```
sudo systemctl reset-failed oom-watch.service
sudo systemctl restart oom-watch.service
sudo systemctl status oom-watch.service
```

Won’t recur because.

- `install-and-verify.sh` step 3 calls `systemctl reset-failed` **before** every restart, so the deployer cannot itself be blocked by a prior throttle.
- The unit’s `StartLimitBurst=3` is intentionally low: three failed starts in a minute means *something is wrong with the daemon, not transient*, and an operator needs to investigate. The restart limit is a feature, not a bug.

---

§4 atop missing

Symptom. Daemon exits immediately with `atop binary not found in PATH`. `systemctl status` shows code=1.

One-command diagnosis.

```
command -v atop || echo "atop NOT installed"
```

Remediation.

```
# ALT Linux:
sudo apt-get install atop
# Debian/Ubuntu:
sudo apt install atop
# RHEL/Fedora:
sudo dnf install atop
# After install:
sudo systemctl restart oom-watch.service
```

Won’t recur because.

- `challenges/challenge-no-atop.sh` runs the daemon under an empty PATH and asserts non-zero exit with `atop` and `not found` in stderr — if a future change ever silently degraded to a /proc fallback, this Challenge fails immediately.
- The systemd unit does NOT have `Wants=atop.service`, intentionally — atop is invoked as a one-shot subprocess; we don’t depend on atop’s own daemon being up.

---

§5 Sandbox blocking atop (`atop sample failed: atop exit -1`)

Symptom. Daemon is `active (running)` but every sample fails. Journal shows `atop sample failed err="atop exit -1: "` (note the empty stderr after the colon — characteristic of signal kill).

One-command diagnosis.

```
sudo dmesg --since "5 minutes ago" | grep -iE 'audit.*seccomp|denied|killed'
```

A line mentioning `comm="atop"` plus `seccomp` or `denied` confirms the kernel killed atop because of a sandbox restriction. If `dmesg` is empty, the cgroup OOM-killed atop instead — check `Memory:` in `systemctl status oom-watch.service` (peak vs. max).

Remediation.

If the cause is seccomp / capabilities:

```
# Pull the current unit which has the relaxed sandbox:
cd /path/to/OOM-Protect && git pull
sudo make oomwatch-install
sudo systemctl daemon-reload
sudo systemctl restart oom-watch.service
```

If the cause is the cgroup memory cap (rare — the daemon plus atop typically peak around 50 MiB), bump `MemoryMax` in `/etc/systemd/system/oom-watch.service.d/local.conf`:

```
[Service]
MemoryMax=256M
MemoryHigh=128M
```

Then `sudo systemctl daemon-reload && sudo systemctl restart oom-watch.service`.

Won't recur because.

- The shipped unit file's comments explicitly call out which directives were dropped (`SystemCallFilter`, `RestrictNamespaces`, `CapabilityBoundingSet`, `SystemCallArchitectures`) and the symptom (`atop exit -1:` with empty stderr) — a future contributor re-enabling them sees the warning before redeploying.
- `challenges/challenge-real-atop.sh` runs the actual atop binary against the daemon and asserts a non-zero memory ratio + a real process name appears in the report, so a sandbox change that silently breaks atop fails the Challenge.

---

§6 Daemon running, no reports (host is calm — expected)

Symptom. Daemon healthy for hours/days, `/var/log/oom-watch/reports/` empty (or only a single startup `*-notice.md` from `-one-shot`).

This is correct behaviour. A monitoring daemon writing nothing means nothing has crossed a threshold. The defaults (`memory_used_ratio_warn=0.90`, `memory_used_ratio_critical=0.95`, `psi_mem_full_avg10_warn=10.0`) are tuned to fire only when the host is genuinely under stress.

Confirm the daemon is iterating.

```
journalctl -u oom-watch.service --since '1 hour ago' --no-pager | grep -E 'verdict|incid-
        ent' | tail -5
```

You should see periodic `verdict severity=OK` lines if log_level is `debug`, or no lines if `info` (the default — verdicts at OK level are not logged to avoid spam). Either way, an absence of `atop sample failed` or `incident write failed` errors means it is healthy.

Force a diagnostic to prove the report path works.

```
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -one-shot
ls -lt /var/log/oom-watch/reports/ | head -3
```

Produces a NOTICE-level forensic report on demand.

---

§7 Daemon running, no reports (thresholds wrong)

Symptom. You believe the host has been under stress (you saw a near-OOM, or atop interactive shows pressure) but no report ever fires.

One-command diagnosis.

```
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -print-config
```

Compare the displayed `thresholds.*` values to the conditions you expected to trip. A `memory_used_ratio_warn=0.95` tuning, for example, will not fire until the host is at 95 % memory, which on a 64 GiB box means leaving only 3 GiB available — by then PSI is usually screaming.

Remediation. Lower the relevant threshold. Recommended starting values for tighter detection:

```
{
  "thresholds": {
    "memory_used_ratio_notice":   0.70,
    "memory_used_ratio_warn":     0.85,
    "memory_used_ratio_critical": 0.92,
    "psi_mem_full_avg10_warn":    5.0,
    "psi_mem_full_avg10_critical": 20.0
  }
}
```

Validate then restart:

```
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run
sudo systemctl restart oom-watch.service
```

Won't recur because. Tuning is a per-host operational choice, not a regression. The ordering invariants (`notice ≤ warn ≤ critical`) are enforced by `config.Validate()` and tested in `internal/config/config_test.go`.

---

§8 Reports filling the disk

Symptom. `df /var/log` approaches full; `find /var/log/oom-watch/reports -name '*.md' | wc -l` returns a large number.

One-command diagnosis.

```
du -sh /var/log/oom-watch/reports && find /var/log/oom-watch/reports -name '*.md' -mtime +30 | wc -l
```

Remediation. Cron a cleanup (default policy: keep 30 days):

```
sudo install -m 0755 /dev/stdin /etc/cron.daily/oom-watch-cleanup <<'EOF'
#!/bin/sh
find /var/log/oom-watch/reports -type f -name '*.md' -mtime +30 -delete
find /var/log/oom-watch/reports -type f -name '*.md.gz' -mtime +90 -delete
EOF
```

Or a logrotate snippet at `/etc/logrotate.d/oom-watch`:

```
/var/log/oom-watch/reports/*.md {
    daily
    rotate 30
    compress
    missingok
    notifempty
    nocreate
    nosharedscripts
}
```

Won't recur because. The daemon does not currently rotate; this is intentional (the daemon's job is to write reports, not manage them). Adding rotation is a per-host operations choice.

---

§9 Cross-UID build (`error obtaining VCS status: exit status 128`)

Symptom. `make oomwatch-build` or `make oomwatch-deploy` fails at the `go build` step with the literal error message above. Common when the repo lives on an external mount owned by your user but you ran `make` as root.

One-command diagnosis.

```
ls -ld $(pwd) && id
```

If the directory's owner UID does not match `id -u`, you have the cross-UID configuration that triggers git's `dubious ownership` safety feature.

Remediation. Already fixed in code. Pull the latest:

```
git pull
sudo make oomwatch-deploy
```

The Makefile passes `-buildvcs=false` to every `go build`; `challenges/lib.sh` exports `GOFLAGS=-buildvcs=false` for every Challenge. Both are belt-and-suspenders against this whole class of failure.

If you genuinely want VCS stamping (you almost never do):

```
git config --global --add safe.directory /path/to/OOM-Protect
# Then remove -buildvcs=false from Makefile + lib.sh.
```

Won't recur because. The flag is in committed code; the only way to reintroduce the failure is to actively delete it.

---

§10 systemd directive drift (`Unknown key 'StartLimitIntervalSec' in section [Service]`)

Symptom. Journal contains:

- `Unknown key 'StartLimitIntervalSec' in section [Service], ignoring`
- `Failed to parse ProtectControlGroups=read-only, ignoring: Invalid argument`
- (or similar warnings about a directive systemd does not recognise)

The warnings are non-fatal — systemd ignores the offending directive and proceeds — but the unit may behave incorrectly (no restart limit, unprotected cgroups, etc.) until fixed.

One-command diagnosis.

```
systemd-analyze verify /etc/systemd/system/oom-watch.service
```

Outputs nothing if the unit is clean; otherwise prints the line numbers and reasons.

Remediation. Pull the current unit (the shipped version is correct for systemd ≥ 230 and ≤ current latest):

```
git pull
sudo make oomwatch-install
sudo systemctl daemon-reload
sudo systemctl reset-failed oom-watch.service
sudo systemctl restart oom-watch.service
```

Won't recur because. The unit file's inline comments name the symptom for each previously-broken directive (`StartLimitIntervalSec`, `ProtectControlGroups`) so a future contributor moving them back sees the warning. `systemd-analyze verify` is part of the pre-merge checklist.

---

§11 Stale installed config (auto-remediation triggered)

Symptom. `make oomwatch-deploy` step 2b prints:

```
[deploy] ERROR INSTALLED CONFIG IS INVALID …
[deploy] auto-remediation: backing up the broken file to /etc/oom-watch/con-
fig.json.broken.<ts>
[deploy] auto-remediation: installing shipped example over the broken config
[deploy] OK fresh example installed and passes -dry-run
[deploy] WARN if you had custom thresholds, re-apply them from the backup
```

You had custom thresholds in `/etc/oom-watch/config.json` and the deployer replaced the file because validation failed (probably after you edited it manually and made a typo, or the example shipped with an unknown field that the parser rejected).

Remediation. Diff the backup against the new config and re-apply your customisations:

```
sudo diff /etc/oom-watch/config.json.broken.* /etc/oom-watch/config.json | less
sudo nano /etc/oom-watch/config.json    # cherry-pick your custom thresholds
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run
sudo systemctl restart oom-watch.service
```

Keep the `.broken.<ts>` file until you have re-applied everything, then optionally delete it:

```
sudo rm /etc/oom-watch/config.json.broken.*
```

Won't recur because. Backup + replace is the *recovery* path, not a bug. The reason the config was invalid is what to chase — usually a deliberate edit that introduced a typo (caught by `DisallowUnknownFields`), or an upgrade from a version with a different schema. Validate before saving in future:

```
sudo nano /etc/oom-watch/config.json
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -dry-run \
    || echo "STOP — fix before restarting daemon"
```

---

§12 Anything else — use the diagnostic bundler

For any failure not covered above, capture every relevant piece of state in a single log file:

```
sudo make oomwatch-diagnose
```

Produces `/tmp/oomwatch-diagnose-<timestamp>.log` with 16 sections: timestamp, host, tool versions, repo state, shipped example SHA, installed paths, full installed config, dry-run verdict, unit state, full unit file, last 200 journal lines, reports directory listing, live atop sample, full `make oomwatch-deploy` run with all step headers, post-deploy unit state, post-deploy journal.

Read it yourself first — most issues are obvious from §6 (config) + §7 (dry-run) + §10 (journal) + §13 (deploy steps). If you need help, paste the file's contents into your support channel.

The bundler is documented in detail in **`oom-watch-deployment-guide.md` §6a**.

---

§13 Re-deployment after upgrade

When new commits land upstream (new features, sandbox tweaks, threshold tuning, parser fixes for new atop versions), re-deploy with:

```
cd /path/to/OOM-Protect
sudo make oomwatch-deploy SUDO=""    # if running as root via su
# or:
sudo -E make oomwatch-deploy
```

For a fully automated update-and-redeploy pipeline:

```
sudo bash oom-watch/scripts/install-and-verify.sh --pull --rebuild
```

The deployer is idempotent: re-running it converges the host to the deployed state regardless of starting point. Custom configs are preserved (via auto-remediate's backup if validation fails); the unit is restarted cleanly with a `reset-failed` so prior throttle state is cleared.

After every upgrade, run:

```
make challenges
```

and confirm `7 / 7 PASS`. A failed Challenge after an upgrade is the canary for a regression — do not declare the upgrade done until they all pass on the target host.

Definition of done for any incident

Per Constitution Article I, an incident is not closed until:

1. The remediation is verified — the daemon is `active (running)`, journal is clean, a fresh report (or forced `-one-shot`) appears on disk.
2. If the incident corresponds to a class of bug that could recur, a Challenge in `challenges/` (or a unit test in `oom-watch/internal/.../*_test.go`) reproduces the failure and asserts the fix. The Constitution forbids “fixed but not tested.”
3. The relevant runbook section above is updated if the symptom or remediation changed.

The runbook is a living document — when you hit a scenario it doesn’t cover, write it up before declaring done.

See also

- **`oom-watch-manual.md`** — feature reference: every config key, severity ladder, report anatomy, threshold tuning recipes.
- **`oom-watch-deployment-guide.md`** — install procedure, prerequisite list, full deployer step-by-step, the diagnostic bundler.
- **`reports/oom-watch-architecture.md`** — design rationale for every directive choice in the systemd unit, the parser, the threshold engine.
- **`Constitution.md`** — Article I (anti-bluff testing), Article II (idempotency), Article III (doc parity).
- **`reports/Crash_Report.md`** — the original 2026-04-28 incident this entire toolkit answers.